Desktopová kuchařka

Dokumentace maturitní práce

Vít Holovka

školní rok: 2025/2026 třída: 8.V

Zadání maturitní práce:

Cílem práce je vytvořit desktopovou aplikaci pro správu kuchařských receptů. Uživatel si v aplikaci může ukládat jednotlivé recepty, přidávat k nim potřebné informace, jako jsou ingredience, postup přípravy či časová náročnost, a následně s nimi dále pracovat. Aplikace umožňuje přehledné zobrazení a vyhledávání receptů, jejich úpravu i mazání. Hlavním cílem projektu je usnadnit práci s uloženými a strukturovanými daty, zajistit jejich dlouhodobé uchování v lokální databázi a nabídnout uživateli jednoduché prostředí pro správu vlastního digitálního kuchařského archivu. Projekt bude napsán v jazyce Python.

Teoretická část - Relační databáze a práce s daty v aplikacích

Prohlašuji, že jsem na práci pracoval samostatně pouze za pomoci použitých zdrojů a že v práci i v dokumentaci jasně vymezuji, které části kódů jsou mým originálním dílem, které jsou upravenou verzí a které jsou převzaty v plném rozsahu.

---

podpis žáka

Obsah

1. TEORETICKÁ ČÁST - RELAČNÍ DATABÁZE 5

1.1. Vlastnosti ACID 5
1.2. Návrh databáze a vztahy - Typy vztahů a cizí klíče 5
1.3. Návrh databáze a vztahy - Normalizace 5
1.4. Jazyk SQL - Rozdělení příkazů 5
1.5. Jazyk SQL - Spojování tabulek (JOIN) 6
1.6. Práce s daty v aplikacích - Přímý přístup vs. ORM 6
1.7. Práce s daty v aplikacích - Bezpečnost: SQL Injection 6
1.8. Výkon a optimalizace - Indexování 6
1.9. Výkon a optimalizace - Uzamykání (souběžnost) 6

2. CÍLE PRÁCE 6

3. ZPŮSOBY ŘEŠENÍ A POUŽITÉ POSTUPY 7

3.1. Celková struktura programu 7
3.2. Inicializace databáze a návrh tabulky 7
3.3. Vkládání a aktualizace receptů 8
3.4. Vyhledávání podle více parametrů 8
3.5. Validace vstupu při ukládání 9
3.6. Ukládání obrázků receptů 10
3.7. Přepínání obrazovek a řídící logika aplikace 10
3.8. Vybírání receptu a předání dat do detailu 11
3.9. Podpora světlého a tmavého režimu 12
3.10. Řešení posouvání obsahu 12
3.11. Testování a ošetření chyb 12

4. PROGRAMOVACÍ PROSTŘEDKY 13

5. ZHODNOCENÍ DOSAŽENÝCH VÝSLEDKŮ 13

6. INSTALACE 13

7. OVLÁDÁNÍ 13

8. SEZNAM POUŽITÝCH INFORMAČNÍCH ZDROJŮ 14

1. TEORETICKÁ ČÁST - RELAČNÍ DATABÁZE

Základním stavebním kamenem je relace, reprezentovaná tabulkou.

- Tabulka (Entita): Objekt reálného světa (např. Uživatel, Objednávka).
- Řádek (Záznam): Konkrétní instance (např. konkrétní uživatel).
- Sloupec (Atribut): Vlastnost entity s definovaným datovým typem (např. jméno, věk).

Každý záznam v tabulce jednoznačně identifikuje Primární klíč (Primary Key - PK).

1.1. Vlastnosti ACID

Relační databáze podporují transakce (skupiny operací), které dodržují pravidla ACID:

- Atomicity (Nedělitelnost): Transakce se provede buď celá, nebo vůbec.
- Consistency (Konzistence): Databáze přechází vždy jen z jednoho platného stavu do druhého.
- Isolation (Izolovanost): Souběžné transakce se navzájem neovlivňují.
- Durability (Trvalost): Potvrzená data jsou trvale uložena i při výpadku.

1.2. Návrh databáze a vztahy - Typy vztahů a cizí klíče

Vztahy mezi tabulkami se definují pomocí Cizího klíče (Foreign Key - FK), který odkazuje na primární klíč jiné tabulky:

- 1:1 (Jedna ku jedné): Jeden záznam odpovídá právě jednomu záznamu v druhé tabulce.
- 1:N (Jedna ku mnoha): Jeden záznam má vazbu na více záznamů jinde (např. jeden zákazník má více objednávek).
- M:N (Mnoha ku mnoha): Záznamy mají vícenásobné vazby obousměrně. Řeší se vytvořením třetí, tzv. vazební tabulky.

1.3. Návrh databáze a vztahy - Normalizace

Proces pro odstranění duplicit a zajištění integrity dat:

- 1NF: Buňky tabulky obsahují pouze jednu nedělitelnou hodnotu.
- 2NF: Splňuje 1NF a všechny neklíčové atributy plně závisí na celém primárním klíči.
- 3NF: Splňuje 2NF a žádný sloupec nezávisí na jiném neklíčovém sloupci (sloupce závisí pouze na primárním klíči).

1.4. Jazyk SQL - Rozdělení příkazů

- DDL (Definice): Tvorba a úprava struktury databáze (`CREATE, ALTER, DROP`).
- DML (Manipulace): Samotná práce s daty (`INSERT, UPDATE, DELETE`).
- DQL (Dotazování): Získávání a filtrování dat (`SELECT, WHERE, GROUP BY`).

1.5. Jazyk SQL - Spojování tabulek (JOIN)

`JOIN` umožňuje kombinovat data z více tabulek do jednoho výsledku:

- `INNER JOIN`: Vrátí pouze záznamy, které mají shodu v obou tabulkách.
- `LEFT JOIN`: Vrátí vše z levé tabulky a k nim odpovídající záznamy z pravé (pokud neexistují, doplní `NULL`).

1.6. Práce s daty v aplikacích - Přímý přístup vs. ORM

- Přímý přístup (SQL v kódu): Nabízí maximální výkon a kontrolu. Nevýhodou je nutnost ručně mapovat databázové řádky na objekty v aplikaci.
- ORM (Object-Relational Mapping): Knihovny (např. Entity Framework) mapují data automaticky. Zrychlují vývoj, ale skrývají SQL dotazy, což může vést k poklesu výkonu u složitých operací.

1.7. Práce s daty v aplikacích - Bezpečnost: SQL Injection

SQL Injection je útok, kdy uživatel podvrhne vstup tak, aby změnil strukturu SQL dotazu. Zcela se mu předchází používáním parametrizovaných dotazů (Prepared Statements), které striktně oddělují logiku SQL příkazu od samotných uživatelských dat.

1.8. Výkon a optimalizace - Indexování

Indexy fungují jako rejstřík v knize – umožňují databázi extrémně rychle vyhledat data bez procházení celé tabulky. Dramaticky zrychlují čtení (`SELECT`), ale mírně zpomalují zápis (`INSERT`, `UPDATE`), protože s každou změnou dat se musí aktualizovat i index.

1.9. Výkon a optimalizace - Uzamykání (souběžnost)

Při souběžné úpravě stejných dat více uživateli se využívají dva přístupy:

- Pesimistické zamykání: Záznam se uzamkne ve chvíli, kdy jej uživatel začne číst pro úpravu. Ostatní musí počkat.
- Optimistické zamykání: Data se neuzamykají předem, ale při ukládání se přes časové razítko nebo verzi zkontroluje, zda je mezitím neupravil někdo jiný.

2. CÍLE PRÁCE

Hlavním cílem práce bylo vytvořit funkční desktopovou aplikaci, která bude sloužit jako osobní digitální kuchařka. Uživatel musí mít možnost recepty pohodlně ukládat, později je vyhledat, upravit nebo odstranit. Chtěl jsem, aby aplikace byla přehledná i pro běžného uživatele, který nemusí rozumět databázím ani programování.

Dalším cílem bylo procvičit si práci s relační databází v praxi. Nestačilo tedy jen vytvořit grafické rozhraní, ale bylo potřeba navrhnout i databázovou strukturu, vyřešit komunikaci mezi uživatelským rozhraním a databází a ošetřit nejčastější chyby při ukládání dat.

Důležitým cílem bylo také to, aby aplikace nefungovala pouze v základní podobě, ale nabízela i některé uživatelsky příjemné prvky. Do projektu proto patří například tmavý a světlý režim, vyhledávání podle více parametrů, ukládání obrázků receptů nebo automatická práce se štítky.

3. ZPŮSOBY ŘEŠENÍ A POUŽITÉ POSTUPY

3.1. Celková struktura programu

Program je rozdělen do více souborů, aby byl přehlednější a jednotlivé části se daly snadno upravovat. Vstupním bodem je soubor `main.py`, který při spuštění nejprve inicializuje databázi a potom vytvoří hlavní okno aplikace.

```
from database import init_db
from components.ui_app import DittoApp

if __name__ == "__main__":
    init_db()
    app = DittoApp()
    app.mainloop()
```

Tento úsek je důležitý hlavně tím, že databáze se připraví ještě před zobrazením rozhraní. Pokud by databáze nebyla inicializována včas, některé části aplikace by se mohly pokoušet načítat data z neexistující tabulky.

3.2. Inicializace databáze a návrh tabulky

Základ celé aplikace tvoří tabulka recepty. Ta vzniká při prvním spuštění programu v souboru database.py. Kromě samotného vytvoření tabulky je zde vidět i řešení pozdějšího rozšíření databáze pomocí příkazů `ALTER TABLE`, což je užitečné při dalším vývoji projektu.

```
def init_db():
    os.makedirs("data", exist_ok=True)
    os.makedirs(IMG_DIR, exist_ok=True)

    conn = _connect()
    c = conn.cursor()
    c.execute("""
        CREATE TABLE IF NOT EXISTS recepty (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nazev TEXT NOT NULL,
            cas TEXT,
            suroviny TEXT,
            postup TEXT,
            cas_min INTEGER,
            image_path TEXT,
            tags TEXT
        )
    """)
```

V tabulce ukládám jak textovou podobu času, tak i číselnou hodnotu `cas_min`. Číselná hodnota je důležitá pro vyhledávání podle času, protože s ní lze snadno porovnávat. Zároveň je zde sloupec `image_path`, který uchovává cestu k obrázku, a sloupec `tags`, do kterého se ukládají štítky.

Zajímavou částí je i migrace starších dat. Pokud se v databázi objeví starší záznamy, které mají vyplněný pouze textový čas, program se pokusí automaticky převést čas na celé minuty.

```
rows = c.fetchall()
for rid, cas_txt in rows:
    if cas_txt is None:
        continue
    m = re.search(r"\d+", str(cas_txt))
    if m:
        c.execute("UPDATE recepty SET cas_min=? WHERE id=?", (int(m.group(0)), rid))
```

3.3. Vkládání a aktualizace receptů

Pro práci s databází jsou připraveny samostatné funkce. Tím se odděluje logika databáze od grafického rozhraní. Funkce `add_recept` vloží nový záznam a vrátí jeho identifikátor. Funkce `update_recept` slouží k úpravě existujícího receptu.

```
def add_recept(nazev, cas_min, suroviny, postup, image_path=None, tags=None):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    cas_min = int(cas_min) if str(cas_min).strip() != "" else None
    cas_txt = f"{cas_min}" if cas_min is not None else ""

    c.execute(
        "INSERT INTO recepty (nazev, cas, cas_min, suroviny, postup, image_path, tags) "
        "VALUES (?, ?, ?, ?, ?, ?, ?)",
        (nazev, cas_txt, cas_min, suroviny, postup, image_path, tags),
    )
    conn.commit()
    rid = c.lastrowid
    conn.close()
    return rid
```

Použité parametrizované dotazy jsou důležité z hlediska správnosti i bezpečnosti. Program neskládá SQL příkaz prostým spojováním textu, ale předává hodnoty odděleně.

3.4. Vyhledávání podle více parametrů

Jednou z nejdůležitějších funkcí celé aplikace je vyhledávání. Uživatel může hledat nejen podle názvu, ale i podle času přípravy, surovin nebo štítků. Logika vyhledávání je soustředěna ve funkci `search_recepty`.

```
def search_recepty(text, mode="nazev", time_cmp="le"):
    conn = _connect()
    c = conn.cursor()

    if mode == "cas":
        try:
            val = int(text)
        except ValueError:
            conn.close()
            return []
```

```
        if time_cmp == "eq":
            c.execute(
                "SELECT id, nazev, cas_min FROM recepty WHERE cas_min = ? ORDER BY id DESC",
                (val,),
            )
        else:
            c.execute(
                "SELECT id, nazev, cas_min FROM recepty WHERE cas_min <= ? ORDER BY id DESC",
                (val,),
            )
```

Na tomto úryvku je vidět, že vyhledávání podle času je řešeno jinak než textové hledání. Program nejprve ověří, zda uživatel zadal číslo, a potom podle nastaveného režimu použije buď přesnou shodu, nebo porovnání „menší nebo rovno“.

```
    else:
        pattern = f"%{text}%"
        if mode == "suroviny":
            c.execute(
                "SELECT id, nazev, cas_min FROM recepty WHERE suroviny LIKE ? ORDER BY id DESC",
                (pattern,),
            )
        elif mode == "tags":
            c.execute(
                "SELECT id, nazev, cas_min FROM recepty WHERE tags LIKE ? ORDER BY id DESC",
                (pattern,),
            )
        else:
            c.execute(
                "SELECT id, nazev, cas_min FROM recepty WHERE nazev LIKE ? ORDER BY id DESC",
                (pattern,),
            )
```

Textové vyhledávání využívá operátor `LIKE` a znaky procenta. Díky tomu stačí zadat část slova a program vrátí i recepty, které obsahují jen část hledaného textu, což je pro běžného uživatele je to pohodlnější.

3.5. Validace vstupu při ukládání

Velmi důležitá je část, která ověřuje správnost údajů zadaných uživatelem. Tato logika se nachází v okně pro přidání nebo úpravu receptu. Nejde jen o technický detail, ale o důležitou ochranu databáze před chybnými daty.

```
def save(self):
    nazev = self.name_entry.get().strip()
    cas = self.time_entry.get().strip()
    suroviny = self.ing_text.get("1.0", "end").strip()
    postup = self.proc_text.get("1.0", "end").strip()
    tags = ", ".join([t.strip().lower() for t in self.tags_entry.get().split(",") if t.strip()])

    if not nazev:
        messagebox.showwarning("Chyba", "Název je povinný.")
        return

    if cas != "":
        try:
            cas_val = int(cas)
        except ValueError:
            messagebox.showwarning("Chyba", "Čas musí být číslo.")
            return
```

Program zde nejprve načte hodnoty ze vstupních polí a následně je kontroluje. Povinný je alespoň název receptu. U času se ověřuje, zda jde o celé číslo. Dále program kontroluje, aby čas nebyl záporný nebo nepřiměřeně vysoký.

Po úspěšné validaci se recept buď vloží jako nový záznam, nebo se upraví již existující. Pokud je k receptu vybraný obrázek, program ho uloží do samostatné složky a cestu k němu zapíše do databáze.

```
if self.recept_id is None:
    rid = add_recept(nazev, cas_val, suroviny, postup, image_path=None, tags=tags)

    img_final = None
    if self.image_path:
        img_final = save_image_for_recipe(rid, self.image_path)

    update_recept(rid, nazev, cas_val, suroviny, postup, image_path=img_final, tags=tags)
```

Tento dvoukrokový postup je potřeba proto, že název obrázku obsahuje identifikátor receptu. Ten ale známe až po uložení záznamu do databáze. Nejprve se tedy vytvoří recept, získá se jeho `id` a teprve potom se uloží obrázek na správné místo.

3.6. Ukládání obrázků receptů

Kromě textových dat jsem do programu přidal i práci s obrázky. Ty se neukládají přímo do databáze jako binární data, ale jako soubory v adresáři `data/images`. V databázi je potom pouze cesta k souboru.

```
def save_image_for_recipe(rid, source_path):
    if not source_path:
        return None

    os.makedirs(IMG_DIR, exist_ok=True)

    ext = os.path.splitext(source_path)[1].lower()
    if ext not in [".png", ".jpg", ".jpeg", ".webp", ".gif"]:
        ext = ".png"

    dest = os.path.join(IMG_DIR, f"recept_{rid}{ext}")
```

Toto řešení je praktičtější než ukládání obrázků přímo do databáze. Databáze zůstává menší a práce se soubory je jednodušší. Zároveň je zde kontrola přípony souboru a ošetření situace, kdy zdroj a cíl ukazují na stejné místo.

3.7. Přepínání obrazovek a řídící logika aplikace

Celá aplikace je postavena na více stránkách, mezi kterými se uživatel pohybuje. Hlavní řídicí logika je soustředěna ve třídě `DittoApp`. Ta vytváří jednotlivé obrazovky a zajišťuje jejich přepínání.

```
for F in (MainMenu, SearchPage, DatabasePage, RecipePage):
    frame = F(container, self)
    self.frames[F.__name__] = frame
    frame.grid(row=0, column=0, sticky="nsew")
```

Tímto způsobem se při spuštění vytvoří všechny důležité stránky programu. Jsou uloženy do slovníku podle názvu třídy, takže je lze jednoduše vyvolat podle jména.

```
def show_frame(self, page_name):
    frame = self.frames[page_name]
    if page_name == "DatabasePage":
        frame.refresh()
    if page_name == "SearchPage":
        frame.refresh_all()
    frame.tkraise()
```

Metoda `show_frame` kromě samotného přepnutí zároveň zajišťuje aktualizaci dat. Například při vstupu do databázové stránky se seznam receptů znovu načte. Tím se uživateli vždy zobrazí aktuální stav po přidání nebo úpravě receptu.

3.8. Vybírání receptu a předání dat do detailu

Po výběru receptu ze seznamu je potřeba předat jeho identifikátor do stránky s detailem. To je vyřešeno metodou `open_recept` ve vyhledávací stránce.

```
def open_recept(self):
    rid = self._get_selected_id()
    if rid is None:
        return

    recipe_page = self.controller.frames["RecipePage"]
    recipe_page.back_target = "SearchPage"
    recipe_page.set_recept(rid)
    self.controller.show_frame("RecipePage")
```

Tato část je důležitá tím, že odděluje seznam receptů od samotného detailu. Stránka s detailem nepracuje přímo s výběrem v listboxu, ale pouze dostane identifikátor receptu a podle něj si sama načte data z databáze. To je čistší návrh a usnadňuje další úpravy.

```
def set_recept(self, rid):
    data = get_recept(rid)
    if data is None:
        self.title_label.config(text="Recept nenalezen")
        ...
        return

    rid_db, nazev, cas, suroviny, postup, *rest = data
    cas_min = rest[0] if len(rest) > 0 else None
    image_path = rest[1] if len(rest) > 1 else None
    tags = rest[2] if len(rest) > 2 else ""
```

Metoda `set_recept` ukazuje, jak se načtený databázový záznam rozebere na jednotlivé hodnoty a následně zobrazí v uživatelském rozhraní. Vedle textových údajů se řeší i načtení obrázku a štítků. Pokud recept neexistuje, zobrazí se uživateli odpovídající zpráva.

3.9. Podpora světlého a tmavého režimu

Do aplikace jsem přidal také světlý a tmavý režim. Uživatel si tak může zvolit vzhled, který mu více vyhovuje. Nastavení je uloženo do konfiguračního souboru, takže po dalším spuštění programu zůstane zachováno.

```
def toggle_theme(self):
    if self.theme == LIGHT_THEME:
        self.theme = DARK_THEME
        self.config_data["theme"] = "dark"
    else:
        self.theme = LIGHT_THEME
        self.config_data["theme"] = "light"

    save_config(self.config_data)
    self.apply_theme()
```

3.10. Řešení posouvání obsahu

V některých částech aplikace je obsah delší než plocha okna. Proto bylo potřeba vyřešit spolehlivé posouvání obsahu myší na různých operačních systémech. Pomocná funkce v souboru `utils.py` zajišťuje podporu jak pro Linux, tak pro Windows nebo touchpad.

```
def bind_scroll_recursive(widget, canvas):
    def _on_mousewheel(event):
        if hasattr(event, "num"):
            if event.num == 4:
                canvas.yview_scroll(-1, "units")
            elif event.num == 5:
                canvas.yview_scroll(1, "units")
            return

        if hasattr(event, "delta") and event.delta:
            canvas.yview_scroll(int(-1 * (event.delta / 240)), "units")

    widget.bind("<MouseWheel>", _on_mousewheel)
    widget.bind("<Button-4>", _on_mousewheel)
    widget.bind("<Button-5>", _on_mousewheel)
```

Tato část je dobrým příkladem drobného, ale důležitého technického problému. Posouvání pomocí kolečka myši totiž není v Tkinteru na všech platformách stejné. Funkce proto ošetřuje oba způsoby generování událostí.

3.11. Testování a ošetření chyb

Aplikaci jsem testoval průběžně během vývoje. Ověřoval jsem hlavně přidávání receptů, jejich úpravu, mazání, vyhledávání podle různých parametrů, přepínání motivu a práci s obrázky. Testování probíhalo především manuálně, protože u GUI aplikací je tento způsob velmi důležitý.

Při vývoji jsem narazil i na několik problémů, například s pozdějším doplněním nových sloupců do databáze, s ukládáním obrázků nebo s tmavým režimem. Tyto chyby jsem postupně odstranil. Dokumentace proto nezachycuje pouze výsledný stav, ale i to, že projekt během vývoje procházel postupným zlepšováním.

4. PROGRAMOVACÍ PROSTŘEDKY

Program je napsán v jazyce Python, který je vhodný pro rychlý vývoj desktopových aplikací. Jeho výhodou je přehledná syntaxe a dostupnost knihoven. Pro grafické rozhraní jsem použil knihovnu Tkinter, která je součástí standardní distribuce Pythonu a je proto snadno dostupná.

Pro databázovou vrstvu jsem použil SQLite, protože je jednoduchá na použití a nevyžaduje instalaci samostatného serveru. Pro práci s obrázky je v programu volitelně využita knihovna Pillow. Pokud není dostupná, aplikace funguje i bez náhledu obrázků, což zvyšuje její použitelnost.

Program jsem vyvíjel na systému Linux v aplikaci Visual Studio Code, testován byl jak na Linuxu tak na Windows.

5. ZHODNOCENÍ DOSAŽENÝCH VÝSLEDKŮ

Domnívám se, že se podařilo splnit hlavní cíle práce. Výsledkem je funkční aplikace, která umožňuje ukládání receptů do lokální databáze, jejich přehledné zobrazení, vyhledávání, úpravu a mazání. Oproti úplnému základu jsem do programu přidal i některé rozšiřující prvky, například ukládání obrázků, práci se štítky, tmavý a světlý režim a validaci vstupních dat.

Za silnou stránku považuji hlavně propojení databáze s grafickým rozhraním a snahu udržet projekt rozdělený do více souborů. Díky tomu je kód přehlednější než řešení v jednom velkém souboru. Uživatel také nemusí pracovat přímo s databází, protože všechny operace jsou zabalené do jednoduchého rozhraní.

Naopak za možné slabší místo považuji to, že databázový model je zatím poměrně jednoduchý. Do budoucna by bylo možné aplikaci rozšířit například o samostatné tabulky kategorií, pokročilejší filtrování, export receptů nebo víceuživatelský režim.

6. INSTALACE

Pro spuštění aplikace je potřeba mít nainstalovaný Python 3. Program využívá standardní knihovny a databázi SQLite, která je součástí Pythonu. Volitelnou závislostí je knihovna Pillow, pokud má aplikace zobrazovat náhledy obrázků.

Při prvním spuštění se automaticky vytvoří složka `data`, databázový soubor a složka pro obrázky. Aplikaci lze spustit příkazem:

```
python main.py
```

7. OVLÁDÁNÍ

Po spuštění aplikace se zobrazí hlavní menu. Z něj se uživatel může přesunout buď do vyhledávání, nebo do databáze receptů. V databázové části lze zobrazit všechny uložené recepty, otevřít detail, vytvořit nový recept, upravit nebo smazat existující.

Při vytváření receptu uživatel vyplní název, čas přípravy, suroviny a postup. Volitelně může přidat i štítky a obrázek. Vyhledávání umožňuje filtrovat recepty podle názvu, času, surovin nebo štítků. Díky tomu je práce s větším počtem receptů pohodlnější.

8. SEZNAM POUŽITÝCH INFORMAČNÍCH ZDROJŮ

What is a Relational Database? – IBM Technology

- https://youtu.be/OqjJjpjDRLc?si=gz98eYD6xOUffl90

SQL explained in 100 seconds – Fireship

- https://youtu.be/zsjvFFKOm3c?si=4fBbQBMdL3MnbQXt

SQLite Databases With Python – freeCodeCamp.org

- https://youtu.be/byHcYRpMgI4?si=cOpnTQHx__SLtwGN

SQLite documentation

- https://www.sqlite.org/docs.html

Python documentation

- https://docs.python.org/3/

Tkinter documentation

- https://docs.python.org/3/library/tkinter.html